Introduction to Operating System

Operating System (or shortly OS) is software that primarily provides services for running applications on a computer system.

The most fundamental system program is the operating system, whose job is to control all the computer's resources and provide a base upon which the application program can be run. **Operating system acts as an intermediary between a user of a computer and the computer hardware.** A computer system can be divided roughly into four components: **the hardware, the operating system, the application program, and the users** as shown in the fig below.

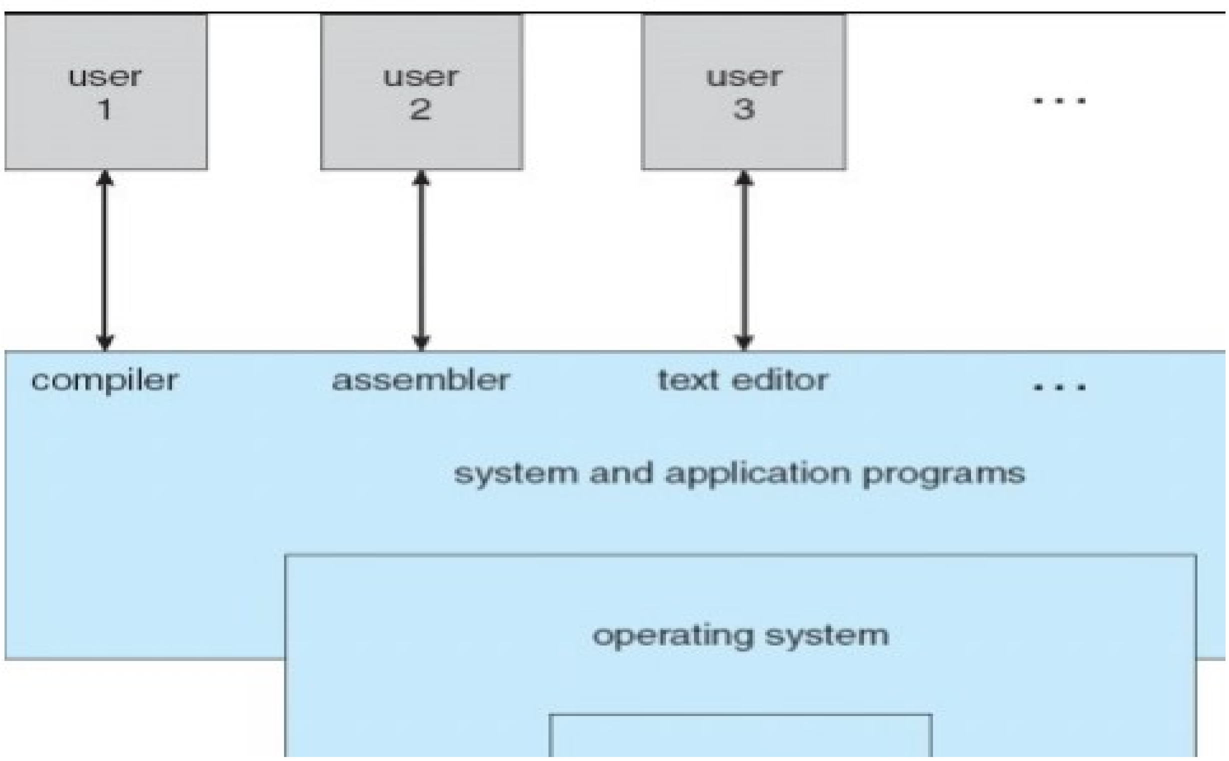


Fig: Abstract view of the components of a Computer System.

It simply provides an environment within which other programs can do useful work.

Applied Operating System

An applied operating system refers to an operating system that is designed and implemented for specific real-world applications or scenarios, as opposed to a theoretical or generic operating system.

Applied operating systems are tailored to meet the specific requirements and constraints of particular environments, industries, or use cases. These requirements could include performance optimization, security enhancements, specialized hardware support, or unique functionality needed for specific applications.

For example, an applied operating system might be designed for use in embedded systems, where the emphasis is on minimal resource usage, real-time responsiveness, and reliability. Another example could be an operating system customized for use in aerospace systems, where stringent safety and reliability standards are essential.

Applied operating systems often involve modifications or extensions to existing operating systems or the development of entirely new operating systems from scratch. They may incorporate features such as specialized device drivers, custom scheduling algorithms, security enhancements, or tailored user interfaces to meet the needs of the target application.

Overall, applied operating systems play a crucial role in enabling various technological advancements by providing the foundation for specialized computing environments and applications.

Goals of Operating System

An operating system is a large and complex system that can only be created by partitioning into small parts. These pieces should be a well-defined part of the system, carefully defining inputs, outputs, and functions.

Although Windows, Mac, UNIX, Linux, and other OS do not have the same structure, most operating systems share similar OS system components, such as file, memory, process, I/O device management.

Following are the components of an operating system:

1. Process Management
2. File Management
3. Network Management
4. Main Memory Management
5. Secondary Storage Management
6. I/O Device Management
7. Security Management

1 **Process management:** The process management component is a procedure for managing many processes running simultaneously on the operating system. Every running software application program has one or more processes associated with them.

Functions of process management

Here are the following functions of process management in the operating system, such as:

- Process creation and deletion.
- Suspension and resumption.
- Synchronization process
- Communication process

2 File management:

Function of file management

The operating system has the following important activities in connection with file management:

- File and directory creation and deletion.
- For manipulating files and directories.
- Mapping files onto secondary storage.
- Backup files on stable storage media.

3 Network Management

Network management is the process of administering and managing computer networks. It includes performance management, provisioning of networks, fault analysis, and maintaining the quality of service.

Functions of Network management

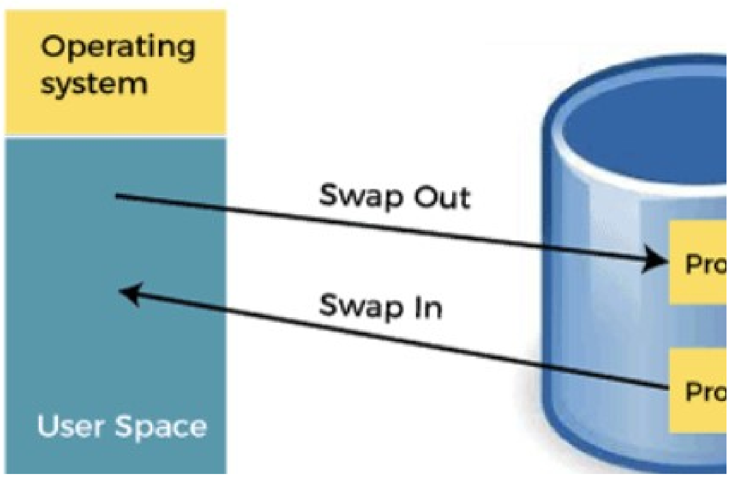
Network management provides the following functions, such as:

1. A distributed system offers the user access to the various resources the network shares.
2. It helps to access shared.

4. Memory Management

Main memory is a large array of storage or bytes, which has an address. The memory management process is conducted by using a sequence of reads or writes of specific memory addresses.

Main memory offers fast storage that can be accessed directly by the CPU. It is costly and hence has a lower storage capacity. However, for a program to be executed, it must be in the main memory.



Functions of Memory management

An Operating System performs the following functions for Memory Management in the operating system:

- It helps you to keep track of primary memory.
- Determine what part of it are in use by whom, what part is not in use.
- In a multiprogramming system, the OS decides which process will get memory and how much.
- Allocates the memory when a process requests.

- It also de-allocates the memory when a process no longer requires or has been terminated.

5. Secondary-Storage Management

The most important task of a computer system is to execute programs. These programs help you to access the data from the main memory during execution. This memory of the computer is very small to store all data and programs permanently. The computer system offers secondary storage to back up the main memory.

Functions of Secondary storage management

Here are some major functions of secondary storage management in the operating system:

- Storage allocation
- Free space management
- Disk scheduling

6. **I/O device management**

One of the important uses of an operating system that helps to hide the variations of specific hardware devices from the user.

Functions of I/O management

The I/O management system offers the following functions, such as:

- It offers a buffer caching system.
- It provides general device driver code.
- It provides drivers for particular hardware devices.

7. Security Management

The various processes in an operating system need to be secured from other activities. Therefore, various mechanisms can ensure those processes that want to operate files, memory, CPU, and other hardware resources should have proper authorization from the operating system.

For example, Memory addressing hardware helps you to confirm that a process can be executed within its own address space. The time ensures that no process has control of the CPU without releasing it.

Lastly, no process is allowed to do its own I/O, to protect, which helps you to keep the integrity of the various peripheral devices.

Computer System Organization and Architecture

Computer Organization and Architecture is used to design computer systems. Computer Architecture is considered to be those attributes of a system that are visible to the user like addressing techniques, instruction sets, and bits used for data, and have a direct impact on the logic execution of a program, It defines the system in an abstract manner, It deals with What does the system do.

Whereas, Computer Organization is the way in which a system has to structure and it is operational units and the interconnections between them that achieve the architectural specifications, It is the realization of the abstract model, and It deals with How to implement the system.

In this Computer Organization and Architecture Tutorial, you'll learn all the basic to advanced concepts like pipelining, micro programmed control, computer architecture, instruction design, and format.

Operating System Structure

Monolithic system architecture

It is the oldest architecture of the operating system. We know that all the core software components of the operating system are collectively known as the **kernel**.

The kernel can access all the resources present in the system. In the **monolithic systems,** each component of the operating system is contained within the kernel. In monolithic architecture, **user services** and **kernel services** are implemented under same address space. It increases the size of the kernel, thus increases size of operating system as well.

All the basic services of OS like **process management, file management, memory management, exception handling, process communication etc.** are all present inside the kernel only.

Linux is a good example of monolithic kernel.

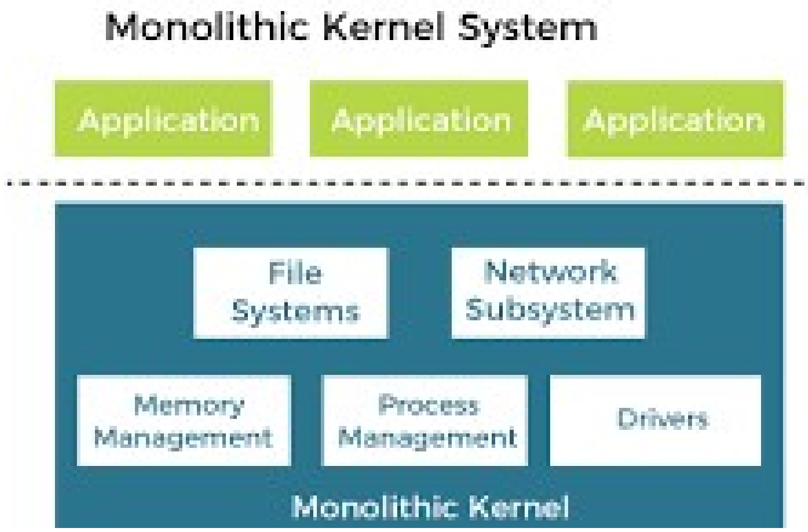


Fig: monolithic architecture

Advantages of Monolithic Kernel –

- This type of operating system has a simple structure. All the components needed for processing are embedded into the kernel.
- All the components can directly communicate with each other and also with the kernel.
- Faster execution due to direct access to all the services.

Disadvantages of Monolithic Kernel –

- One of the major disadvantages of monolithic kernel is that, if anyone service fails it leads to entire system failure.
- Monolithic OS has more tendencies to generate errors and bugs. The reason is that user processes use same address locations as the kernel.

- If user has to add any new service. User needs to modify entire operating system.

Layered Structure of Operating System

The operating system is split into various layers. In the layered operating system and each of the layers have different functionalities. This type of operating system was created as an improvement over the early monolithic systems. The **Microsoft Windows NT Operating System** is a good example of the layered structure.

Why Layering in Operating System?

Layering provides a distinct advantage in an operating system. All the layers can be defined separately and interact with each other as required. Also, it is easier to create, maintain and update the system if it is done in the form of layers. Change in one-layer specification does not affect the rest of the layers.

Each of the layers in the operating system can only interact with the layers that are above and below it. The lowest layer handles the hardware and the uppermost layer deals with the user applications.

Layers in Layered Operating System

There are six layers in the layered operating system. A diagram demonstrating these layers is as follows:

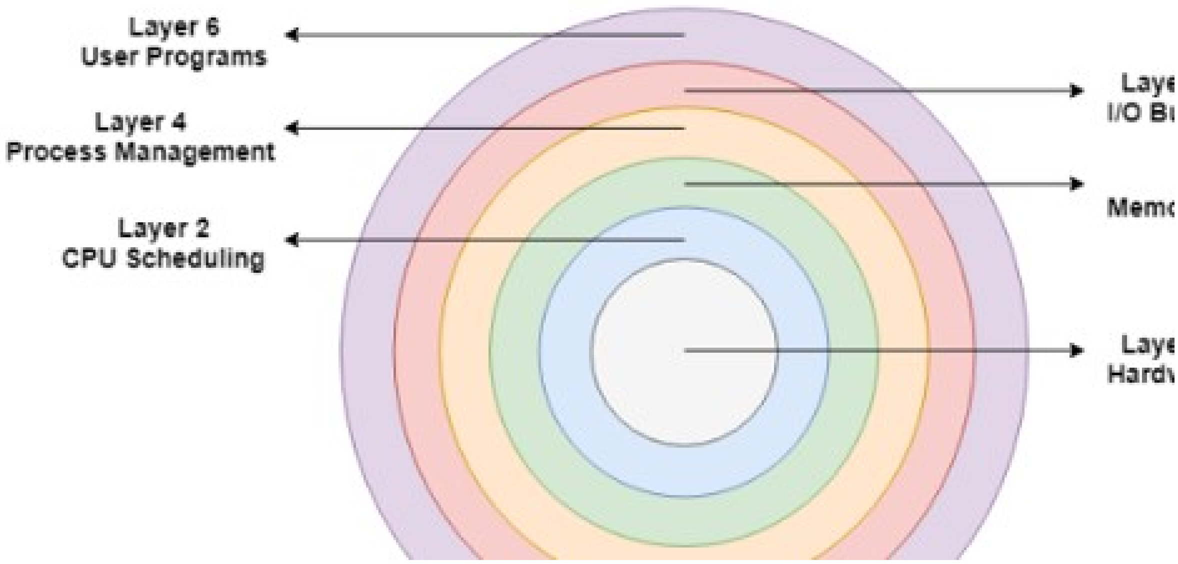


Figure: Layered architecture of OS

Details about the six layers are:

Hardware

This layer interacts with the system hardware and coordinates with all the peripheral devices used such as printer, mouse, keyboard, scanner etc. The hardware layer is the lowest layer in the layered operating system architecture.

CPU Scheduling

This layer deals with scheduling the processes for the CPU. There are many scheduling queues that are used to handle processes. When the processes enter the system, they are put into the job queue. The processes that are ready to execute in the main memory are kept in the ready queue.

Memory Management

Memory management deals with memory and the moving of processes from disk to primary memory for execution and back again. This is handled by the third layer of the operating system.

Process Management

This layer is responsible for managing the processes i.e. assigning the processor to a process at a time. This is known as process scheduling. The different algorithms used for process scheduling are FCFS (first come first served), SJF (shortest job first), priority scheduling, round-robin scheduling etc.

I/O Buffer

I/O devices are very important in the computer systems. They provide users with the means of interacting with the system. This layer handles the buffers for the I/O devices and makes sure that they work correctly.

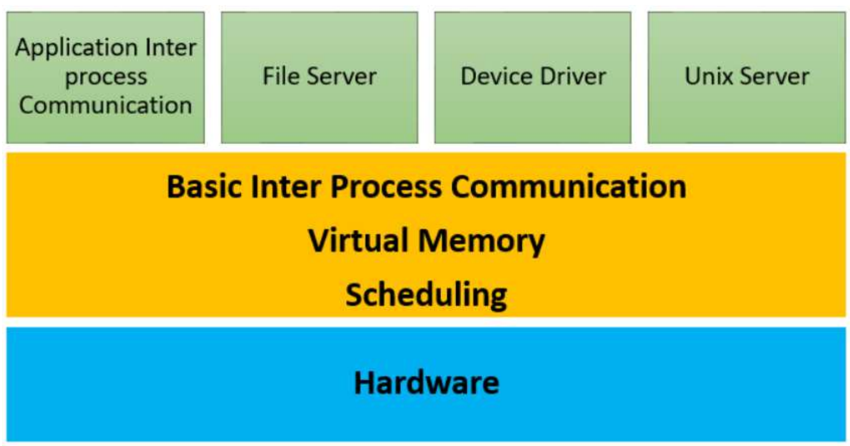
User Programs

This is the highest layer in the layered operating system. This layer deals with the many user programs and applications that run in an operating system such as word processors, games, browsers etc.

Micro-kernel structure

This structure designs the operating system by removing all non-essential components from the kernel and implementing them as system and user programs. This result in a smaller kernel called the micro-kernel.

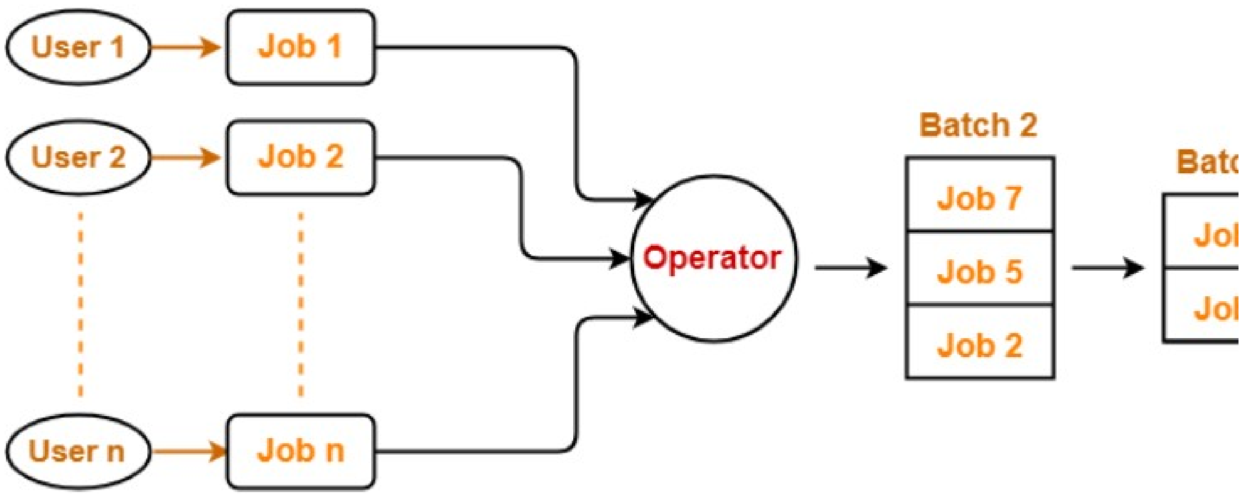
Advantages of this structure are that all new services need to be added to user space and does not require the kernel to be modified. Thus, it is more secure and reliable as if a service fails then rest of the operating system remains untouched. Mac OS is an example of this type of OS.



Types of Operating System

1. **Batch Operating system**

   The user who uses a batch operating system does not interact with the computer directly. Each user prepares its job on an off-line device like punch cards and submits it to the computer operator. To speed up the processing, jobs with similar needs are batched together and run as a group. Once the programmers have left their programs with the operator, they sort the programs with similar needs into batches.



The batch operating system grouped jobs that perform similar functions. These job groups are treated as a batch and executed simultaneously. A computer system with this operating system performs the following batch processing activities:

1. A job is a single unit that consists of a preset sequence of commands, data, and programs.
2. Processing takes place in the order in which they are received, i.e., **first come, first serve**.
3. These jobs are stored in memory and executed without the need for manual information.
4. When a job is successfully run, the operating system releases its memory.

Advantages:

1) Same jobs in the batch have higher execution speed.

2) Once a process completes its execution, next job from job spool gets executed without any user interaction.

4) To speed up the processing speed, the batch process can partition into the number of processes.

Disadvantages:

1. All the jobs of a batch are executed sequentially one after the other.
2. The output is obtained only after all the jobs are executed.
3. Thus, priority cannot be implemented if a certain job has to be executed on an urgent basis.
4. The computer system and the users have no direct interaction.
5. If a job enters an infinite loop, other jobs must wait for an unknown period of time.

2. **Time sharing Operating System/ Multiprogramming**

   Time-sharing is a technique which enables many people, located at various terminals, to use a particular computer system at the same time. Time-sharing is a logical extension of multiprogramming. Processor's time which is shared among multiple users simultaneously is termed as time-sharing.

   Multiple jobs are executed by the CPU by switching between them, but the switches occur so frequently. Thus, the user can receive an immediate response. For example, in a transaction processing, the processor executes each user program in a short burst or quantum of computation. That is, if n users are present, then each user can get a time quantum. When the user submits the command, the response time is in few seconds at most.

The operating system that uses CPU scheduling and multiprogramming to provide each user with a small portion of a time. Computer systems that were designed primarily as batch systems have been modified to time-sharing systems.

Advantages

- Provides the advantage of quick response.
- Reduces CPU idle time.

Disadvantages

- Problem of reliability.
- Question of security and integrity of user programs and data.
- Problem of data communication.

3. Real Time operating System

A real-time system is defined as a data processing system in which the time interval required to process and respond to inputs is so small that it controls the environment. The time taken by the system to respond to an input and display of required updated information is termed as the **response time**. So, in this method, the response time is very less.

Real-time systems are used when there are **rigid time requirements** on the operation of a processor or the flow of data and real-time systems can be used as a control device in a dedicated application. A real-time operating system must have **well-defined, fixed time constraints,** otherwise the system will fail. For example, Scientific experiments, medical imaging systems, industrial control systems, weapon systems, robots, air traffic control systems, etc.

Two types of RTOS systems are:

Hard Real Time:

In Hard RTOS, the deadline is handled very strictly which means that given task must start executing on specified scheduled time, and must be completed within the assigned time duration.

Example: Medical critical care system, Aircraft systems, etc.

Soft Real Time:

Soft Real time RTOS, accepts some delays by the Operating system. In this type of RTOS, there is a deadline assigned for a specific job, but a delay for a small amount of time is acceptable. So, deadlines are handled softly by this type of RTOS.

Example: Online Transaction system and Livestock price quotation System.

Advantages

Focus on Application: – This type of operating system focus on applications which are running and usually give less importance to other application residing in waiting stage of life cycle. So, less applications or tasks are managed and give exact result on current execution work.

Real time operating system in embedded system: – Due to small size of programs RTOS can also be used in embedded systems like in transport and others.

Error Free: – RTOS is error free that mean it has no chances of error in performing tasks.

Memory Allocation: – Memory allocation is best managed in this type of systems.

Disadvantages

Limited Tasks: – There are only limited tasks run at the same time and the concentration of these system are on few applications to avoid errors and other task have to wait. Sometime there is no time limit of how much the waiting tasks have to wait.

Use heavy system resources: – RTOS used lot of system resources which is not as good and is also expensive.

Complex Algorithms: – RTOS uses complex algorithms to achieve a desired output and it is very difficult to write the algorithms for a designer.

Not easy to program: – The designer has to write proficient program for real time operating system which is not easy.

4. Distributed OS

Distributed Operating System is a model where distributed applications are running on multiple computers linked by communications. This system looks to its users like an ordinary centralized operating system but runs on multiple, independent central processing units (CPUs).

These systems are referred as ***loosely coupled systems*** where each processor has its own local memory and processors communicate with one another through various communication lines, such as high-speed buses or telephone lines. By loosely coupled systems, we mean that such computers possess no hardware connections at the CPU – memory bus level, but are connected by external interfaces that run under the control of software.

The Distributed OS involves a collection of autonomous computer systems, capable of communicating and cooperating with each other through a LAN / WAN.

LOCUS and MICROS are the best examples of distributed operating systems.

Advantages

- With resource sharing facility, a user at one site may be able to use the resources available at another.
- Speedup the exchange of data with one another via electronic mail.
- Failure of one site in a distributed system doesn't affect the others, the remaining sites can potentially continue operating.
- Better service to the customers.
- Reduction of the load on the host computer.
- Reduction of delays in data processing.

Disadvantages

- It is difficult to provide adequate security in distributed systems because the nodes as well as the connections need to be secured.
- Some messages and data can be lost in the network while moving from one node to another.
- The database connected to the distributed systems is quite complicated and difficult to handle as compared to a single user system.
- Overloading may occur in the network if all the nodes of the distributed system try to send data at once.

5. Parallel System/ Multi processing

Parallel Processing Systems are designed to speed up the execution of programs by dividing the program into multiple fragments and processing these fragments simultaneously. Such systems are multiprocessor systems also known as tightly coupled systems. Parallel systems deal with the simultaneous use of multiple computer resources that can include a single computer with multiple processors, a number of computers connected by a network to form a parallel processing cluster or a combination of both.

Parallel computing is an evolution of serial computing where the jobs are broken into discrete parts that can be executed concurrently. Each part is further broken down to a series of instructions. Instructions from each part execute simultaneously on different CPUs.

Advantages

1. Parallel computing saves time.
2. Solve Larger Problems in a short point of time.
3. Many problems are so large and/or complex that it is impractical or impossible to solve them on a single computer, especially given limited computer memory, which is possible with parallel system.
4. You can do many things simultaneously by using multiple computing resources.

Disadvantages

1. Parallel solutions are harder to implement, they're harder to debug or prove correct, and they often perform worse than their serial counterparts due to communication and coordination overhead.

2. The extra cost incurred is due to data transfers, synchronization, communication, thread creation/destruction, etc. These costs can sometimes be quite large, and may actually exceed the gains due to parallelization.

6. Networking Operating System

A network operating system (NOS) is a computer operating system (OS) that's designed primarily to support workstations, PCs and, in some instances, older terminals that are connected on a local area network (LAN). The software behind NOS enables multiple devices within a network to communicate and share resources with each other. However, a typical NOS no longer exists, as most OS have built-in network stacks that support a client-server model.

A NOS coordinates the activities of multiple computers across a network. This can include such devices as PCs, printers, file servers and databases connected to a local network. The role of the NOS is to provide basic network services and features that support multiple input requests simultaneously in a multiuser environment.

For example: Microsoft Widows Server, Unix OS, Cisco Internetwork OS, Junos OS

Advantages

- New technology and hardware upgrades can be easily integrated into the system.
- The server can be accessed remotely from different locations and types of systems.
- The centralized server is very stable.
- Security is managed by the server.

Disadvantages

- Most operations depend on the central location.
- The cost of buying and running servers is high.
- It needs regular maintenance and updates.

7. Operating System for Embedded Devices

An embedded operating system is a computer operating system designed for use in embedded computer systems. It has limited features. The term "embedded operating system" also refers to a "real-time operating system". The main goal of designing an embedded operating system is to perform specified tasks for non-computer devices. It allows the executing programming codes that deliver access to devices to complete their jobs.

An embedded operating system is a combination of software and hardware. It produces an easily understandable result by humans in many formats such as **images, text**, and **voice**. Embedded operating systems are developed with programming code, which helps convert hardware languages into software languages like **C** and **C++**.

The embedded operating system improves overall efficiency by controlling all hardware resources and minimizing response times for specific tasks for which devices were built.

Advantages

There are various advantages of an embedded operating system. Some of them are as follows:

1. It is small in size and faster to load.
2. It is low cost.
3. It is easy to manage.
4. It provides better stability.

5. It provides higher reliability.
6. It provides some interconnections.
7. It has low power consumption.
8. It helps to increase the product quality.

Disadvantages

There are various disadvantages of an embedded operating system. Some of them are as follows:

1. It isn't easy to maintain.
2. The troubleshooting is harder.
3. It has limited resources for memory.
4. It isn't easy to take a back of embedded files.
5. You can't change, improve, or upgrade an embedded system once it's been developed.
6. If any problem occurs, you need to reset the setting.
7. Its hardware is limited.

Functions of Operating System/ Operating System Services

An Operating System supplies different kinds of services to both the users and to the programs as well. It also provides application programs (that run within an Operating system) an environment to execute it freely. It provides users the services run various programs in a convenient manner.

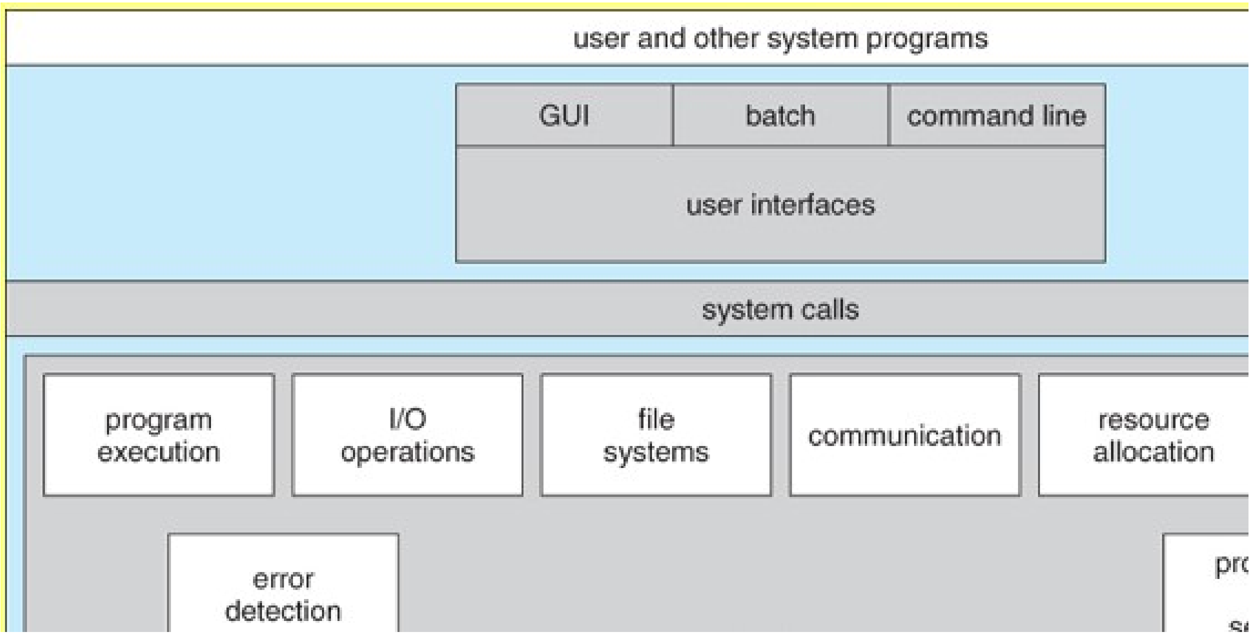


Figure: Services provided by Operating System

An operating system is an interface which provides services to both the user and to the programs. It provides an environment for the program to execute. It also provides users with the services of how to execute programs in a convenient manner. The operating system provides some services to program and also to the users of those programs. The specific services provided by the OS are off course different.

1. User interface

The OS provides a user interface (UI), an environment for the user to interact with the machine. The UI is either graphical or text-based.

A user interface of any operating system can be classified into one of the following types:

a) Graphical user interface (GUI)

b) Command line user interface (CLI)

Graphical user interface (GUI)

The graphical user interface is a type of GUI that enables the users to interact with the operating system by means of point-and-click operations. GUI contains several icons representing pictorial representation of the variables such as a file, directory, and device.

Command line user interface (CLI)

Command line interface is a type of UI that enables the users to interact with the operating system by issuing some specific commands. In order to perform a task in this interface, the user needs to type a command at the command line. When the user enters the key, the command line interpreter receives a command. The CLI is that the user needs to remember a lot to interact with the operating system. Therefore, these types of interfaces are not considered very friendly from the users' perspective.

2. Program Execution

- An operating system must be able to load many kinds of activities into the memory and to run it. The program must be able to end its execution, either normally or abnormally.

- A process includes the complete execution of the written program or code. There are some of the activities which are performed by the operating system:
    - The operating system Loads program into memory
    - It also Executes the program
    - It Handles the program's execution
    - It Provides a mechanism for process synchronization
    - It Provides a mechanism for process communication

3. I/O Operations

- The communication between the user and devices drivers are managed by the operating system.
- I/O devices are required for any running process. In I/O, a file or an I/O device can be involved.
- I/O operations are the read or write operations which are done with the help of input-output devices.
- Operating system give the access to the I/O devices when it required.

4. File system manipulation

A file system is a collection of directories for easy understand and usage. These directories contain some files. There are some major activities which are performed by an operating system with respect to file management.

- The operating system gives an access to the program for performing an operation on the file.
- Programs need to read and write a file.
- The user can create/delete a file by using an interface provided by the operating system.
- The operating system provides an interface to the user creates/ delete directories.
- The backup of the file system can be created by using an interface provided by the operating system.

5. Communication

In the computer system, there are collection of processes which do not share memory, peripherals devices or a clock, the operating system manages communication between all the processes. Multiple processes can communicate with every process through communication lines in the network. There are some major activities that are carried by an operating system with respect to communication.

- Two processes may require data to be transferred between the process.
- Both the processes can be on one computer or a different computer, but are connected through a computer network.

6. Error handling

An error is one part of the system that may cause malfunctioning of the complete system. The operating system constantly monitors the system for detecting errors to avoid some situations.

The error can occur anytime and anywhere. The error may occur anywhere in the computer system like in CPU, in I/O devices or in the memory hardware. There are some activities that are performed by an operating system:

- The OS continuously checks for the possible errors.
- The OS takes an appropriate action to correct errors and consistent computing.

7. Resource management

When there are multiple users or multiple jobs running at the same time resources must be allocated to each of them. There are some major activities that are performed by an operating system:

- The OS manages all kinds of resources using schedulers.
- CPU scheduling algorithm is used for better utilization of CPU.

8. Protection

When several disjoints processes execute concurrently it should not be possible for any process to interfere with another process. Every process in the computer system must be secured and controlled.

System calls:

In computing, a **system call** is the programmatic way in which a computer program requests a service from the kernel of the operating system it is executed on. A system call is a way for programs to **interact with the operating system**. A computer program makes a system call when it makes a request to the operating system's kernel. System call **provides** the services of the operating system to the user programs via **Application Program Interface (API).** It provides an interface between a process and operating system to allow user-level processes to request services of the operating system. System calls are the only entry points into the kernel system. All programs needing resources must use system calls. System calls provide a means for user or application programs to call upon the services of the operating system.

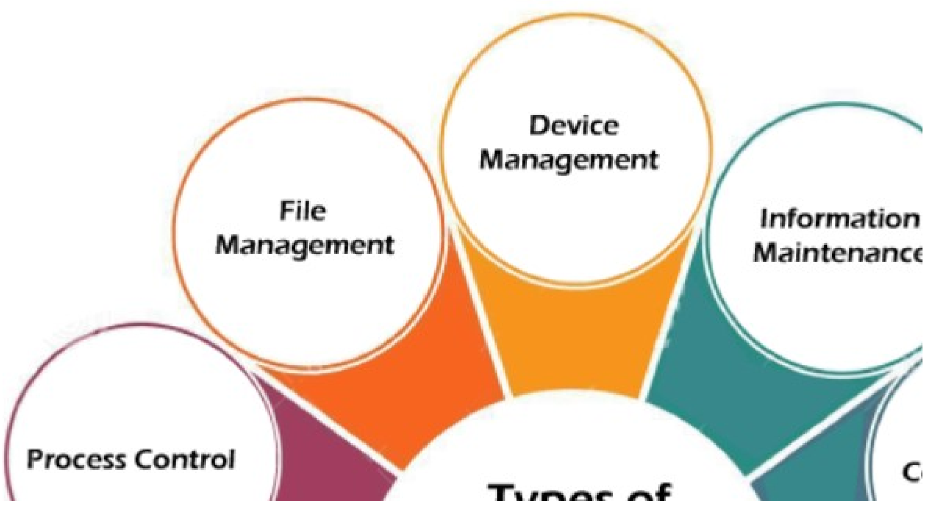
There are various situations where you must require system calls in the operating system. Following of the situations are as follows:

1. It is must require when a file system wants to create or delete a file.
2. Network connections require the system calls to sending and receiving data packets.
3. If you want to read or write a file, you need to system calls.

4. If you want to access hardware devices, including a printer, scanner, you need a system call.
5. System calls are used to create and manage new processes.

Services Provided by System Calls:

1. Process creation and management (Some process control examples include CreateProcess(), ExitProcess(), etc.)
2. File Access, Directory and File system management (Some file management examples include CreateFile(), ReadFile(), WriteFile(), etc.)
3. Information Maintenance (GetCurrentProcessID(), SetTimer(), etc)
4. Device handling(I/O) (Some examples of device management SetConsoleMode(), ReadConsole(), WriteConsole(), etc.)
5. Communication (CreatePipe(), CreateFileMapping())
6. Protection (SetFileSecurity(), InitializeSecurityDescriptor())



Virtual machine structure of operating system

The fundamental idea behind a virtual machine is to abstract the hardware of a single computer (the CPU, memory, disk drive, network interface card, and so forth) into several execution environments, thereby creating an illusion that each separate execution environment is running its own private computer.

In a Virtual Machine - each process "seems" to execute on its own processor with its own memory, devices, etc. The virtual machine idea first appeared **in 1972 in an IBM operating system known as VM running on mainframe computers**. In VM, a high-level layer was provided that altered the users view of the underlying system. This layer made the underlying machine and software layers appear to be a one-user system running a one-user operating system, whose commands and system-services might be different from those of the underlying OS - though the resultant virtual machine still appeared to have the same HARDWARE instruction set. That is, the virtual machine looked like a small member of the same family as the actual processor in use.

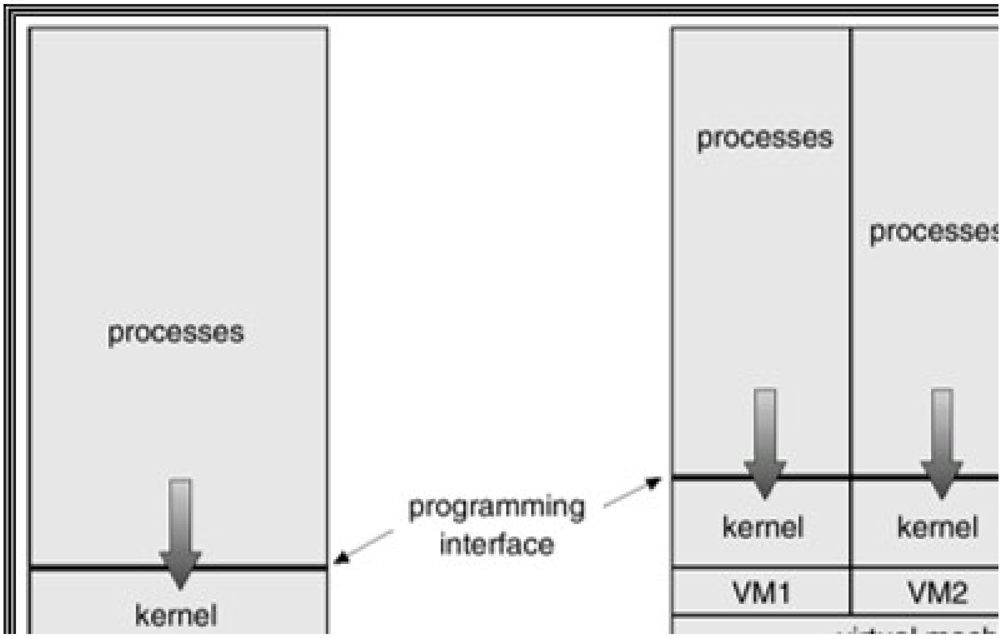


Figure: Virtual machine architecture of OS

Client-server structure of operating system

It is a kind of an operating system where the processes are categorized into servers, each of which provide some service, and the clients which make use of these services. This model is known as the client-server model. Communication between clients and servers is often by message passing. To obtain a service, a client process constructs a message saying what it wants and sends it to the appropriate service. The service then does the work and sends back the answer.

Kernel Data Structures

The kernel data structures in an operating system are essential components used by the kernel (the core part of the operating system) to manage various aspects of the system's operation. These data structures are designed to efficiently organize and manipulate information related to processes, memory, devices, file systems, and other system resources. Here are some common kernel data structures found in operating systems:

1. **Process Control Block (PCB)**: PCB is a data structure used by the kernel to store information about each process in the system. It typically includes details such as process state, process ID (PID), program counter, CPU registers, scheduling information, memory management information, and pointers to other process-related data structures.

2. **Scheduler Data Structures**: These data structures are used by the scheduler component of the kernel to manage the execution of processes on the CPU. Examples include ready queues, which hold processes ready to execute, and scheduling policies such as priority queues, round-robin queues, or multi-level feedback queues.

3. **Memory Management Data Structures**: Kernel memory management data structures manage the allocation and deallocation of memory in the system. This includes data structures such as page tables, which map virtual addresses to physical

addresses, free memory lists, used memory lists, and data structures for tracking memory allocation status (e.g., bitmaps).

4. **File System Data Structures**: File system data structures manage the organization and access of files and directories on storage devices. Examples include inode structures, which represent files and their metadata, directory structures, file allocation tables (FAT), and file control blocks (FCB) used by file system drivers.
5. **Device Management Data Structures**: These data structures are used by the kernel to manage I/O devices such as disks, network interfaces, and peripherals. Examples include device control blocks (DCB) for each device, interrupt request (IRQ) tables, direct memory access (DMA) buffers, and buffers for device I/O operations.
6. **Kernel Data Structures for Synchronization**: Kernel synchronization data structures are used to manage synchronization and concurrency in the system to prevent race conditions and ensure consistency. Examples include mutexes, semaphores, condition variables, spinlocks, and atomic variables.
7. **Networking Data Structures**: In networked operating systems, kernel networking data structures manage network interfaces, sockets, packet buffers, routing tables, and other networking-related information.

8. **Inter-process Communication (IPC) Data Structures**: IPC data structures facilitate communication and synchronization between processes. Examples include message queues, shared memory segments, pipes, and signals.